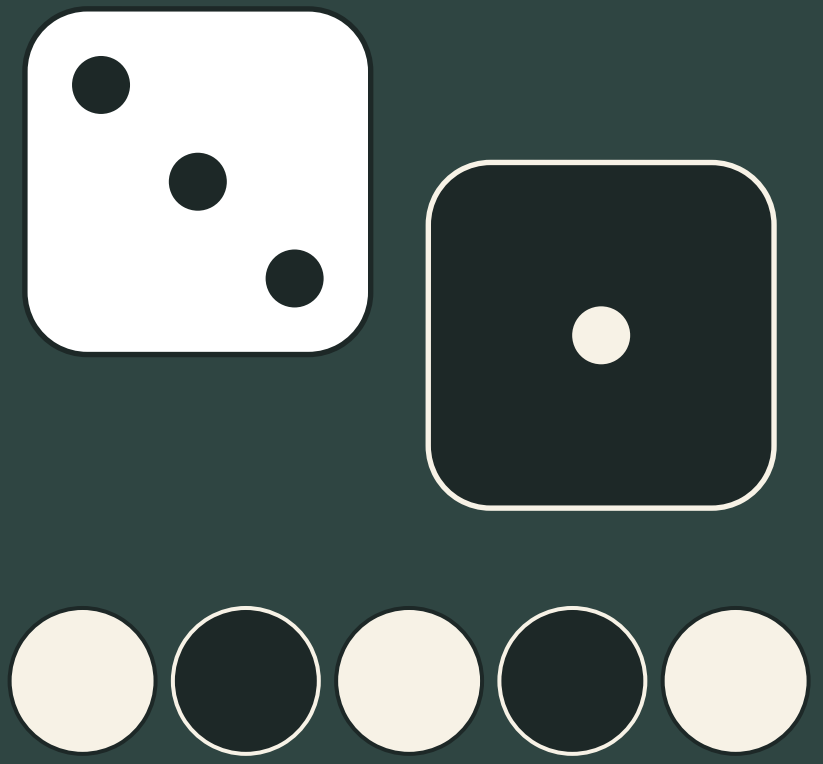
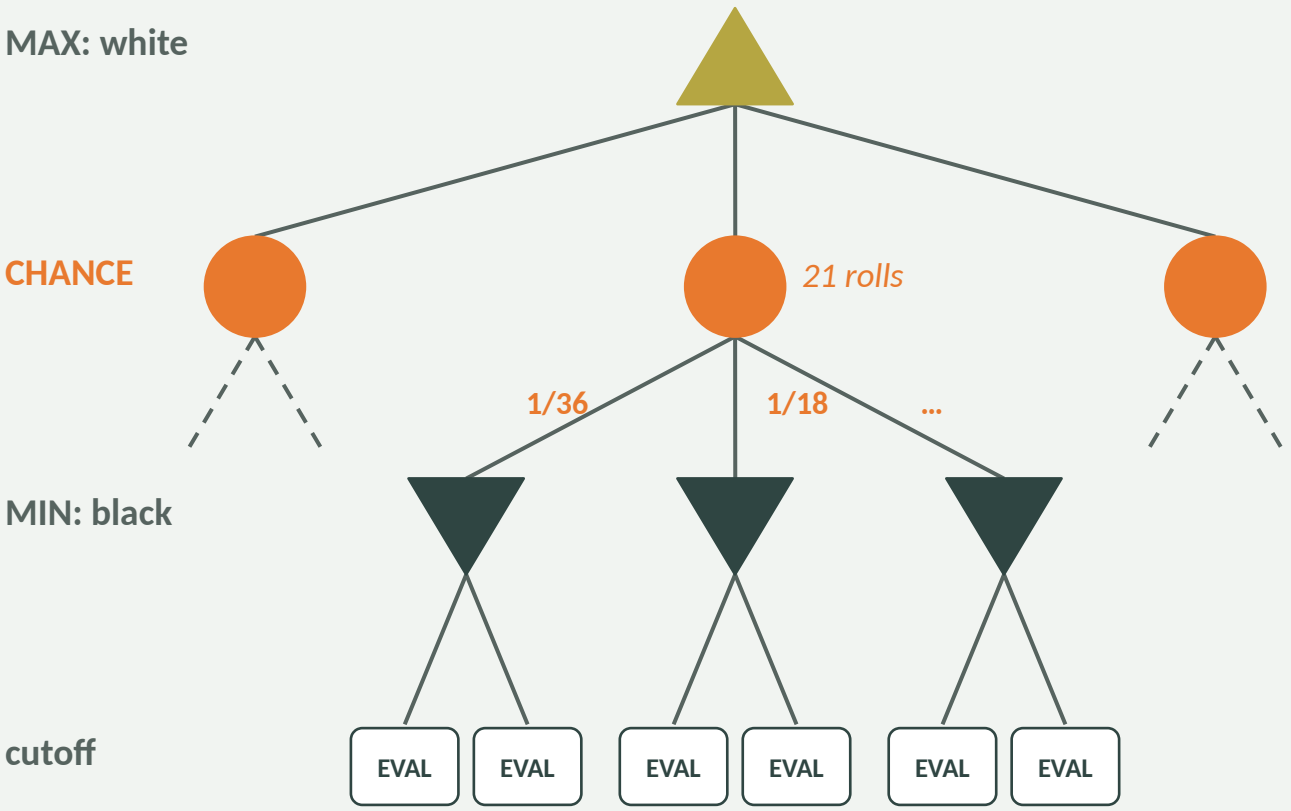
Games with chance

Backgammon in Python

Rules as generators, an EVAL you can tune, and expectiminimax with a cutoff



Where chance enters the tree




```
EXPECTIMINIMAX(n) =
  EVAL(n)          if cutoff(n)
  max over moves   if MAX moves
  min over moves   if MIN moves
  sum of P(r) * value(r)
                   if chance node
```

Course slides M6, 38 to 42. Between two players' decisions there is now a chance node: we cannot choose the roll, only weight it by its probability.

Why we have to cut the search

21

different rolls at every chance node

≈ 21

different moves per roll from the opening (4 to 75)

5,611

positions evaluated to choose one move with 3-1

≈ 2.5 M

if we searched one more chance level

So we do what course slides M6, 31 to 33 propose: search one chance level, then stop and estimate the position with a heuristic EVAL instead of playing the game to the end.

Numbers measured from the opening position; Python and Prolog give the same counts.

A position is a tuple

```
INITIAL = ((-2, 0, 0, 0, 0, 5,  0, 3, 0, 0, 0, -5,
            5, 0, 0, 0, -3, 0, -5, 0, 0, 0, 0, 2), 0, 0, 0, 0)
```

```
>>> from backgammon import *
>>> show(INITIAL)
 13  14  15  16  17  18 |  19  20  21  22  23  24
 W5   .   .   .  B3   . |  B5   .   .   .   .  W2
 B5   .   .   .  W3   . |  W5   .   .   .   .  B2
 12  11  10   9   8   7 |   6   5   4   3   2   1
 bar: W0 B0   off: W0 B0
```

1. **Points**

   24 integers: + white, − black.

2. **Bar**

   Checkers of each side on the bar.

3. **Off**

   Checkers each side has borne off.

4. **Moves**

   (from, to, die), with "bar" and "off".

One checker, one die: a generator

```
def steps(s, d):
    """All moves of one white checker with die d: (move, new_state)."""
    pts, bar_w = s[0], s[1]
(1) if bar_w > 0:                                  # on the bar: enter first
        t = 25 - d
        if pts[t - 1] >= -1:                       # not blocked by 2+ black
            yield ("bar", t, d), apply_move(s, ("bar", t, d))
        return
    home = all_home(s)
    for f in range(1, 25):
        if pts[f - 1] <= 0:                        # need a white checker on f
            continue
        t = f - d
(2)     if t >= 1:
            if pts[t - 1] >= -1:                   # land: not blocked
                yield (f, t, d), apply_move(s, (f, t, d))
(3)     elif home and (t == 0 or not white_above(pts, f)):
            yield (f, "off", d), apply_move(s, (f, "off", d))   # bear off
```

(1) On the bar: enter first, at 25 − die; return stops any other move.

(2) Land on a point with at most one black checker; if there is one, it is hit.

(3) Bear off with an exact die, or a larger one if nothing is above.

Black is white in a mirror

```python
def mirror(s):
    pts, bar_w, bar_b, off_w, off_b = s
    return (tuple(-n for n in reversed(pts)), bar_b, bar_w, off_b, off_w)

def mirror_point(p):
    return p if p in ("bar", "off") else 25 - p

def mirror_move(m):
    return (mirror_point(m[0]), mirror_point(m[1]), m[2])

def single(s, player, d):
    """All moves of one checker of player with die d."""
    if player == "white":
        yield from steps(s, d)
    else:
        for move, w2 in steps(mirror(s), d):
            yield mirror_move(move), mirror(w2)
```

1. Reverse the 24 points and flip the signs: black's position seen as if it were white's.
2. Black moves = white moves on the mirrored board, mirrored back. Only white's rules are written.

Using the dice, one at a time

```
    def dice_orders(dice):
        d1, d2 = dice
(1)     if d1 == d2:
            return [(d1, d1, d1, d1)]                # doubles: four moves
        return [(d1, d2), (d2, d1)]

    def use_dice(s, player, dice):
        """All ways to use the dice in this order: (moves, final)."""
        if not dice:
            yield [], s
            return
        options = list(single(s, player, dice[0]))
(3)     if not options:                              # this die cannot be used
            yield [], s
            return
(2)     for move, s1 in options:
            for moves, final in use_dice(s1, player, dice[1:]):
                yield [move] + moves, final
```

(1) Doubles give four moves; otherwise try both orders of the two dice.

(2) Move one checker with the first die, then use the rest: recursion.

(3) If a die cannot be used, the turn stops there: maybe fewer moves.

The two rules of a whole turn

```
    def legal_turns(s, player, dice):
        """List of (final, moves), one per distinct final position.

        Rules: use as many dice as possible; if only one die can be
        used and they are different, it must be the larger one."""
        found = [(moves, final) for order in dice_orders(dice)
                 for moves, final in use_dice(s, player, order)]
(1)     most = max(len(moves) for moves, _ in found)
        if most == 0:
            return []                                  # no legal move: pass
        found = [(m, f) for m, f in found if len(m) == most]
        d1, d2 = dice
(2)     if most == 1 and d1 != d2:
            bigger = [(m, f) for m, f in found if m[0][2] == max(d1, d2)]
            if bigger:
                found = bigger
(3)     turns = {}
        for moves, final in found:
            turns.setdefault(final, moves)             # one per final position
        return sorted(turns.items(), key=lambda item: item[0])
```

(1) Keep only the turns that use the most dice. None at all means pass.

(2) If only one die fits and they differ, it must be the larger one.

(3) A dictionary keyed by the final position keeps one turn per result.

EVAL: five features, five weights

```python
DEFAULT_WEIGHTS = (1, 0, 4, 3, 2)

def evaluate(s, w=DEFAULT_WEIGHTS):
    won = winner(s)
    if won:
        return 1000 if won == "white" else -1000
    return sum(wi * fi for wi, fi in zip(w, features(s)))
```

Pips	Off	Blots	Home	Bar
1	0	4	3	2
difference in pip count: who is ahead in the race	checkers already borne off	lone checkers the rival could hit	points made in your own home board	checkers stuck on the bar

All features are differences, from white's point of view. Without contact between the sides, only the race (pips) counts.

Expectiminimax with a cutoff, in three pieces

```
    # The 21 distinct rolls: doubles 1/36, the others 2/36
1   ROLLS = [((b, a), (1 if a == b else 2) / 36)
             for a in range(1, 7) for b in range(a, 7)]

2   def best_reply(s, player, dice, w):
        """player rolls dice and plays his best move; we stop there
        and use EVAL instead of searching deeper (the cutoff)."""
        turns = legal_turns(s, player, dice)
        if not turns:
            return evaluate(s, w)
        values = [evaluate(final, w) for final, _ in turns]
        return max(values) if player == "white" else min(values)

3   def chance(s, player, w):
        """The chance node before player rolls."""
        return sum(p * best_reply(s, player, dice, w) for dice, p in ROLLS)
```

1. The chance node's 21 outcomes and their probabilities.
2. The rival plays his best reply, judged by EVAL: here we cut.
3. The chance node is the sum of P(roll) × value.

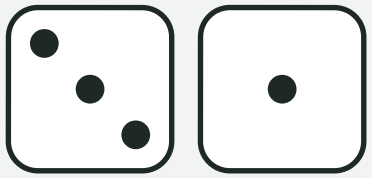
decide: the move with the best expected value

```
def decide(s, player, dice, w=DEFAULT_WEIGHTS, beam=12):
    """Return (value, moves, final). Only the beam best moves by
    EVAL are searched one level deeper."""
    if player == "black":                          # decide in the mirror
        best = decide(mirror(s), "white", dice, w, beam)
        if best is None:
            return None
        value, moves, final = best
        return (-value, [mirror_move(m) for m in moves], mirror(final))
    turns = legal_turns(s, "white", dice)
    if not turns:
        return None
    turns.sort(key=lambda t: evaluate(t[0], w), reverse=True)
    scored = []
    for final, moves in turns[:beam]:
        value = evaluate(final, w) if winner(final) else chance(final, "black", w)
        scored.append((value, moves, final))
    return max(scored, key=lambda t: t[0])
```

1. Black decides in the mirror, as if it were white.
2. Sort by EVAL and keep the beam best (12): a second cut.
3. For each, the chance node of black's roll.
4. Take the move with the highest expected value.

Run it

```
$ python3 -i backgammon.py
>>> len(legal_turns(INITIAL, "white", (3, 1)))
16
>>> value, moves, final = decide(INITIAL, "white", (3, 1))
>>> value, moves
(-0.9166666666666666, [(8, 5, 3), (6, 5, 1)])
```



8/5 6/5: the agent makes its 5-point, the move backgammon books recommend for an opening 3-1. It took about 0.15 seconds.

Same rules, same EVAL as the web demo

4,000

random positions: Python and the page's JavaScript find exactly the same legal moves

2,000

positions: both give exactly the same EVAL value

0

differences, including doubles, the bar, bearing off and turns with no move

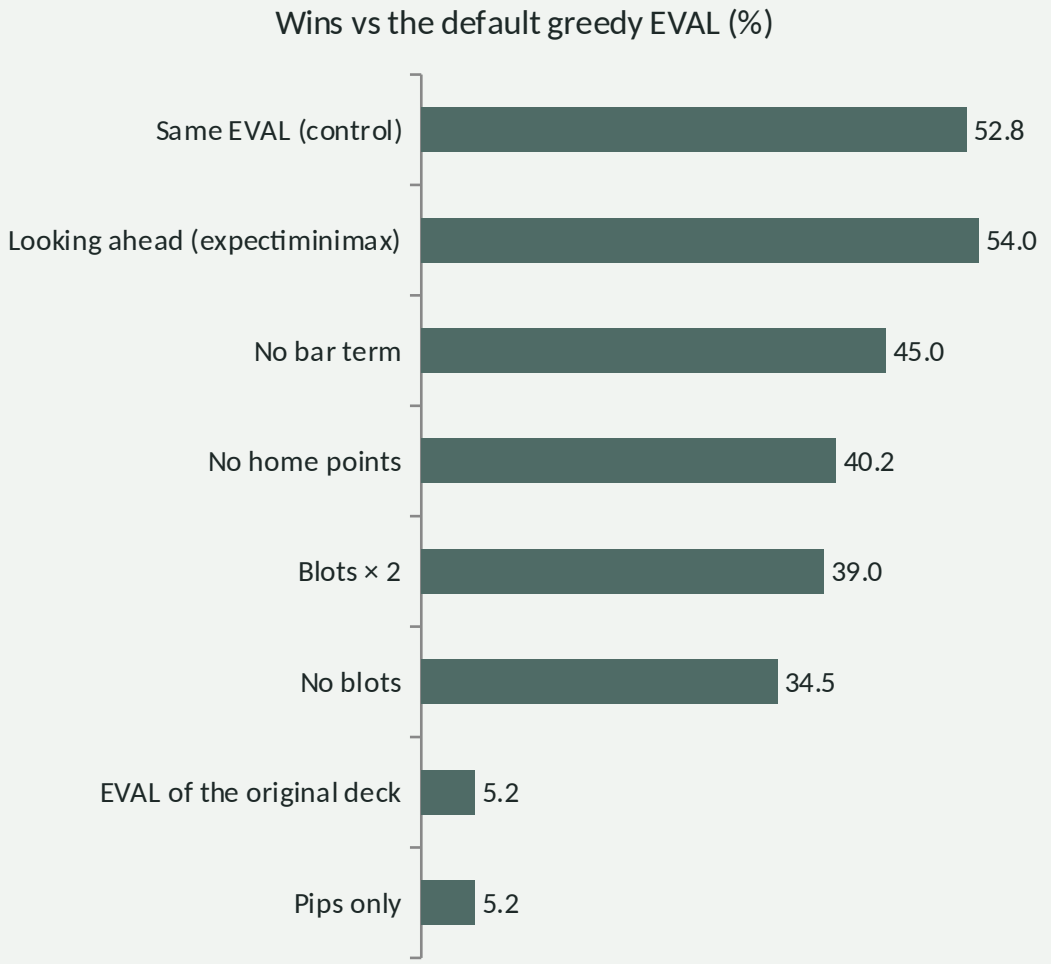
The same check passes for the Prolog version: the three implementations play the same game.

Design your own EVAL

```
>>> tournament(greedy((1, 0, 4, 3, 2)),
...            greedy((1, 0, 0, 0, 0)), 100, seed=2)
95
```

A greedy agent plays the move with the best EVAL, without looking ahead: a fast way to compare weights. 100 games take about a second.

Control ≈ 50%. With 400 games, differences under ±5 points are just luck.



Why black decides in the mirror

Our first control, the same EVAL against itself, gave white 64% of 400 games.

Before

```
turns = legal_turns(s, player, dice)
pick = max if player == "white" else min
return pick(turns,
            key=lambda t: evaluate(t[0], w))[0]
```

After

```
if player == "black":
    return mirror(act(mirror(s), "white", dice))
turns = legal_turns(s, "white", dice)
return max(turns,
           key=lambda t: evaluate(t[0], w))[0]
```

Why: the EVAL gives whole numbers, so many moves tie. max and min break ties by the first move in the list, and that is not the mirror image for the two colors: each side got a different playing style. Deciding in the mirror makes both colors play exactly alike, and the control drops to 51.5%.

Prolog and Python, side by side

Idea	Prolog	Python
A position	`state(Pts, BW, BB, OW, OB)`	`(pts, bar_w, bar_b, off_w, off_b)`
All moves of a checker	`step/4 + backtracking`	`steps(): a generator, yield`
Collect all turns	`findall/3`	`list comprehension`
One per final position	`sort(1, @<, ...)`	`dict keyed by the tuple`
Chance node	`aggregate_all(sum(Pr*V), ...)`	`sum(p * ... for ...)`
100 greedy games	`≈ 8 s`	`≈ 1 s`
One expectiminimax move	`≈ 0.45 s`	`≈ 0.04 s`

Same algorithm, same results, two styles: Prolog describes the moves and lets backtracking find them all; Python generates them explicitly, and runs about ten times faster here.

Predict first, then run

1 Tune one weight

Will doubling the blot weight help? Run a control too.

```
tournament(greedy((1, 0, 8, 3, 2)),
           greedy((1, 0, 4, 3, 2)), 200, seed=1)
```

2 Add a feature

Points made anywhere (2+ checkers). Does it beat the noise?

```
# at the end of features(s), when there is contact:
made_diff = (sum(1 for n in pts if n >= 2)
             - sum(1 for n in pts if n <= -2))
# return it as a sixth feature (0 without contact)
# and add a sixth weight, e.g. (1, 0, 4, 3, 2, 1)
```

3 Is looking ahead worth it?

Expectiminimax vs greedy, same EVAL.

```
tournament(expecti(12, (1, 0, 4, 3, 2)),
           greedy((1, 0, 4, 3, 2)), 200, seed=1)
```

Chance changes the max into an average

One chance level plus a good EVAL is enough to play reasonable backgammon. Which EVAL is good is an experimental question.

21 rolls averaged at every chance node

0.04 s per decision in Python

5% wins of an EVAL that ignores blots and home points